

Install

```
pip install gumbel-mcts
```

```
from gumbel_mcts import PUCT, GumbelDense, GumbelSparse  
from gumbel_mcts import GameLogic, MCTSModel    # protocols (optional, for type hints)
```

Contract 1 – GameLogic

```
class GameLogic(Protocol):
    NUM_ACTIONS: int      # total action space (9 for TicTacToe, 225 for 15×15 Gomoku)
    BOARD_SHAPE: tuple    # single board shape ((3, 3) for TicTacToe)
    MAX_MOVES: int        # max game length before forced draw
    PLAYER_1: int         # 1
    PLAYER_2: int         # 2

    def fast_step(
        self, board: np.ndarray, action: int, player: int
    ) -> tuple[float, int, bool, np.ndarray]:
        # returns: reward, winner (0=none), done, new_board
        ...

    def get_valid_mask(
        self, board: np.ndarray, player: int
    ) -> np.ndarray:    # shape (NUM_ACTIONS,), dtype float32, 1.0 = legal
        ...
```

GameLogic – TicTacToe example

```
class TicTacToeLogic:
    NUM_ACTIONS = 9
    BOARD_SHAPE = (3, 3)
    MAX_MOVES = 9
    PLAYER_1 = 1
    PLAYER_2 = 2

    fast_step = staticmethod(fast_step) # @njit function
    get_valid_mask = staticmethod(get_valid_mask) # @njit function

    @njit(cache=True)
    def fast_step(board, action, player):
        r, c = action // 3, action % 3
        board[r, c] = player
        # ... check rows, cols, diagonals ...
        return reward, winner, done, board

    @njit(cache=True)
    def get_valid_mask(board, player):
        mask = np.zeros(9, dtype=np.float32)
        for i in range(3):
            for j in range(3):
                if board[i, j] == 0:
                    mask[i * 3 + j] = 1.0
        return mask
```

Contract 2 – MCTSModel

```
class MCTSModel(Protocol):
    logic: GameLogic

    def forward_for_mcts(self, batch: dict) -> dict:
        # Input
        #   batch["boards"]          (B, *BOARD_SHAPE) float tensor
        #   batch["current_player"]  (B,)                int tensor
        #
        # Output
        #   {"policy": (B, NUM_ACTIONS), # probabilities, sum to 1
        #    "value":  (B,) or (B, 1)}   # in [-1, 1]
        ...
```

MCTSModel – TinyModel example

```
class TinyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(9, 64), nn.ReLU(), nn.Linear(64, 64), nn.ReLU())
        self.policy_head = nn.Linear(64, 9)
        self.value_head = nn.Linear(64, 1)
        self.logic = TicTacToeLogic() # ← required attribute

    def forward_for_mcts(self, batch):
        h = self.net(batch["boards"]).float()
        policy = torch.softmax(self.policy_head(h), dim=-1)
        value = torch.tanh(self.value_head(h))
        return {"policy": policy, "value": value}
```

PUCT – usage

```
tree = PUCT(n_games=1, max_nodes=500, logic=logic, device="cpu")
tree.initialize_roots([0], board[None], np.array([player]))
tree.run_simulation_batch(model, [0], num_simulations=50)

# pick the most-visited action
visits, q = tree.get_all_root_data(n_active=1)
action = int(np.argmax(visits[0]))
```

GumbelDense – usage

```
tree = GumbelDense(n_games=1, max_nodes=500, logic=logic, device="cpu")
tree.initialize_roots([0], board[None], np.array([player]))
move = tree.run_simulation_batch(model, [0], num_simulations=50)

action = move[0]    # argmax is built-in – Gumbel returns the move directly
```

Full game loop

```
logic = TicTacToeLogic()
model = TinyModel()
model.eval()

board = np.zeros((3, 3), dtype=np.int8)
player = 1

while True:
    tree = GumbelDense(n_games=1, max_nodes=500, logic=logic, device="cpu")
    tree.initialize_roots([0], board[None], np.array([player]))
    move = tree.run_simulation_batch(model, [0], num_simulations=50)
    action = move[0]

    _, winner, done, board = logic.fast_step(board, action, player)

    if done:
        print({0: "Draw", 1: "X wins", 2: "O wins"}[winner])
        break
    player = 3 - player
```

Parallel games – `n_games > 1`

```
N = 8    # run 8 games simultaneously

tree = GumbelDense(n_games=N, max_nodes=5000, logic=logic, device="cuda")
tree.initialize_roots(
    list(range(N)),          # active game indices
    boards,                  # (N, *BOARD_SHAPE)
    np.array(players),       # (N,)
)
moves = tree.run_simulation_batch(model, list(range(N)), num_simulations=50)
# moves[i] is the chosen action for game i
```